Università degli Studi di Messina

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE, SCIENZE FISICHE E SCIENZE DELLA TERRA

Bachelor's Degree in Computer Science L-31

Curriculum: Informatica

LAB-02: Agenti Intelligenti

Supervisore:

Prof. Giacomo Fiumara

Studenti:

Aurora Fabio – 558520

Mario Parisi – 555657

Sandi Russo – 553675

Anno Accademico

2025/2026

Indice

1 Agenti Aspirapolvere in 1D e 2D **1**

1 Attività A 1

1.1 A1 1

1.2 A2 1

1.3 A3 2

2 Attività B 3

2.1 B1 3

2.2 B2 3

2.3 B3 4

2.4 B4 4

2.5 B5 5

3 Attività C 6

3.1 C1 6

3.2 C2 6

3.3 C3 7

4 Mini-Report di Gruppo 10

4.1 Tabella comparativa dei quattro agenti 10

4.2 Discussione sulla razionalità nei diversi contesti 11

4.3 Estensioni possibili 11

Agenti Aspirapolvere in 1D e 2D

Simple reflex, model-based e ambienti parzialmente osservabili

1 Attività A

1.1 A1

L'ambiente 1D a due stanze è **parzialmente osservabile** perché alla funzione `percept` viene passata la posizione attuale e lo stato della cella in cui si trova, vale a dire se è sporca o pulita. Di conseguenza, l'agente non ha alcuna visibilità sul resto dell'ambiente, in quanto non conosce lo stato delle celle adiacenti finché non si sposta su di esse. Questo implica che l'agente non è in grado di pianificare il percorso ottimale a priori, ma deve prendere decisioni locali basandosi esclusivamente sulle informazioni della cella corrente.

1.2 A2

L'agente fornito è un **model-based reflex agent**, poiché mantiene uno stato interno che persiste tra un passo e l'altro e che influenza le decisioni future. Nel codice lo notiamo nella funzione `init_agent_1d`, che inizializza il dizionario contenente il campo `direction`. Questa è un'informazione che l'agente costruisce e conserva autonomamente per ricordare quale direzione stava controllando, senza che tale dato provenga in alcun modo dal percetto.
Nella funzione `agent_1d_policy`, l'agente non si limita a reagire al percetto corrente: legge `direction` dallo stato interno e lo aggiorna quando raggiunge uno degli stati estremi `pos == 0` o `pos == n_squares - 1`, invertendo la direzione di marcia. Questo aggiornamento rappresenta esattamente il meccanismo che distingue un agente model-based da uno simple reflex.
Infine, restituisce il nuovo stato aggiornato insieme all'azione scelta. Nel ciclo di simulazione `run_episode_1d`, questo stato viene propagato ad ogni iterazione tramite la riga `action, agent_state = agent_1d_policy(agent_state, percept)`, garantendo la continuità dell'informazione nel tempo.

1.3 A3

Il comportamento dell'agente *razionale* cambia in base ad alcuni scenari che si basano sulla performance:

- **Scenario 1**: *costo di passo* > *premio* (movimento = -2, pulizia = +1): notiamo che la performance decresce nel tempo, poiché ogni spostamento penalizza l'agente più di quanto una cella pulita lo premi. Un agente razionale dovrebbe, quindi, minimizzare i movimenti, evitando di controllare celle già pulite e cercando di pulire il massimo numero di celle sporche con il minor numero di passi possibile.

- **Scenario 2**: *costo di passo* < *premio* (movimento = -1, pulizia = +5): notiamo che la performance cresce positivamente, poiché il guadagno ottenuto pulendo una cella supera il costo degli spostamenti. In questo caso un agente razionale dovrebbe esplorare aggressivamente tutto l'ambiente per non perdere nessuna cella sporca, poiché ogni cella non pulita rappresenta un'opportunità mancata molto vantaggiosa.

- **Scenario 3**: *costo di passo* = *premio* (movimento = -1, pulizia = +1): notiamo che la performance tende a rimanere stabile, poiché ogni guadagno ottenuto pulendo una cella viene annullato dal costo degli spostamenti necessari per raggiungerla. Un agente razionale dovrebbe pulire solo le celle immediatamente raggiungibili senza spostarsi ulteriormente, poiché un qualsiasi movimento aggiuntivo non produce alcun beneficio netto.

2 Attività B

2.1 B1

Formalmente, l'ambiente 2D è caratterizzato dalle seguenti proprietà:

- **Parzialmente osservabile**: perché la funzione `percept_2d` restituisce all'agente unicamente la sua coordinata attuale (r, c) e lo stato di pulizia di quella specifica cella con il booleano `is_dirty`. L'agente non ha accesso all'intero dizionario durante l'esecuzione, quindi ignora lo stato del resto della griglia fino a quando non esplora fisicamente le altre celle;
- **Deterministico**: il modello di transizione dell'ambiente è completamente prevedibile. Guardando la funzione `step_env_2d`, ogni azione ha un esito certo al 100%. Se l'agente esegue `LEFT` e ha spazio, si sposta sempre a sinistra; se esegue `SUCK` su una cella sporca, questa diventa sempre pulita. Non c'è alcun elemento di probabilità nell'effetto delle azioni;
- **Statico**: l'ambiente cambia stato esclusivamente in risposta alle azioni dell'agente. Se l'agente esegue `NOOP` e non si muove, la disposizione rimane invariata. Non c'è un processo esterno che aggiunge sporco mentre l'agente sta operando;
- **Discreto**: questa proprietà si applica allo stato e allo spazio, dove l'ambiente è diviso in una griglia senza posizioni intermedie continue; al tempo, ove la simulazione avanza a scatti; e alle azioni, dove l'agente sceglie da un set finito di comandi;
- **Singolo agente**: il comportamento di un solo agente influisce sull'ambiente.

2.2 B2

No, **questo agente non utilizza uno stato interno dinamico**, vale a dire non possiede una memoria degli eventi passati; di conseguenza è un **simple reflex agent**. Il dizionario `agent_state` passato alla policy contiene esclusivamente i parametri strutturali dell'ambiente (`rows` e `cols`), statici e invariati durante l'esecuzione; l'agente non memorizza le celle visitate né la direzione di movimento al turno precedente. La politica è definita dalle coordinate assolute ad ogni singolo passo. Invece di ricordare la direzione, l'agente utilizza un approccio puramente matematico sulla posizione corrente:

- se la riga è pari, scansiona verso destra;
- se la riga è dispari, scansiona verso sinistra.

2.3 B3

Per compensare la parziale osservabilità, l'agente mantiene un modello interno `world_belief`.

1. Per l'agente, `world_belief` rappresenta lo stato interno, in cui il robot aspirapolvere annota tutto ciò che sa dell'ambiente fino a quel momento. Visto che l'ambiente è **parzialmente osservabile**, inizialmente `world_belief` è impostato a *unknown*. Man mano che avviene l'esplorazione, questa mappa si popola ed è importantissimo per l'agente affinché non ripassi sulle celle già pulite.

2. Ad ogni interazione con l'ambiente, l'agente riceve una percezione locale dai suoi sensori e tramite questa posizione aggiorna il `world_belief`: modifica lo stato della cella sovrascrivendo `unknown` con `clean` o `dirty`.

2.4 B4

Un agente **simple reflex** seleziona le proprie azioni basandosi esclusivamente sul percetto corrente, ignorando lo *stato precedente*. Nell'ambiente 2D implementato, l'agente affronta una situazione di **osservabilità parziale**, dunque i suoi sensori rilevano unicamente la cella corrente e il suo stato (`dirty` o `clean`). Un agente puramente reattivo funziona correttamente solo se l'ambiente è **pienamente osservabile**.
Se, ad esempio, l'agente percepisce uno stato `clean`, non avendo alcuna memoria, non saprebbe in quale direzione muoversi per trovare ulteriore sporco. Di conseguenza, i loop infiniti sono spesso inevitabili per gli **agenti simple reflex** che operano in ambienti **parzialmente osservabili**.
Il modo più efficace per gestire la **parziale osservabilità** è far sì che l'agente tenga traccia della parte del mondo che non può vedere. L'agente deve mantenere uno **stato interno** che dipende dallo *stato precedente* e che rifletta gli aspetti non osservati dello stato corrente. Nel codice, il concetto di *internal state* è implementato attraverso la funzione `init_agent_2d_model_based`, che inizializza due elementi fondamentali per la sua mappa mentale:

- La variabile `pos`, che tiene traccia della posizione stimata dell'agente.

- Il dizionario `world_belief`, che rappresenta il modello dell'ambiente, classificando costantemente ogni cella come `"unknown"`, `"dirty"`, o `"clean"`.

Grazie allo *stato interno*, l'agente model-based è in grado di agire in modo **razionale**. Quando il percetto gli indica che la cella corrente è `clean`, l'agente non si blocca ma aggiorna il suo `world_belief` e utilizza un algoritmo di ricerca (`bfs_next_step`) per calcolare e intraprendere un percorso verso i target più vicini che risultano ancora `dirty`

o *unknown*. Questa capacità di mantenere, aggiornare e interrogare lo *stato interno*, gli permette di provvedere alla mancanza di un sensore globale, portando a termine l'obiettivo di esplorare e pulire l'intera griglia in modo sistematico.

2.5 B5

Scenario	**Serpente** *perf. media*	**Model-Based** *perf. media*	**Osservazioni qualitative**
(5×5), prob. $= 0.8$	72,75	75,75	Con una presenza elevata di sporco, il guadagno di 10 punti per la pulizia compensa le inefficienze di movimento. Entrambi gli agenti trovano sporco quasi a ogni passo, quindi le performance convergono verso valori molto simili.
(8×3), prob. $= 0.5$	67,75	71	In una griglia stretta e allungata, l'agente model-based supera il serpente grazie all'esplorazione dinamica verso la cella *unknown* più vicina, ottimizzando i movimenti e risparmiando passi. Il serpente è costretto a controllare l'intera larghezza di ogni riga, sprecando punti in movimenti ridondanti.
(12×9), prob. $= 0.1$	30,5	10,5	In una griglia ampia con poco sporco, il serpente supera nettamente il model-based. La traiettoria pre-programmata garantisce copertura totale senza ripassare celle già visitate. L'agente model-based genera invece percorsi frammentati, lasciando zone inesplorate ai bordi e accumulando penalità di -1 per raggiungerle.

Tabella 1.1: Confronto delle performance tra agente serpente e agente model-based in tre scenari distinti.

3 Attività C

3.1 C1

Un **agente 2d con dimensioni note** può inizializzare fin da subito una mappa completa della griglia, sapendo esattamente quante righe e colonne esistono. Conosce a priori i confini dello spazio e non ha bisogno di scoprirli durante l'esecuzione. Il suo stato interno può contenere una struttura dati completa (una matrice) e non deve gestire il concetto di *muro scoperto dinamicamente*.

Un agente con dimensioni ignote, costruisce una mappa sparsa: parte solo dalla posizione iniziale e aggiunge celle alla struttura `visited` man mano che le visita. Non sa dove si trovano i bordi finché non ci sbatte contro, momento in cui aggiunge la cella corrispondente all'insieme `walls`. Per fare questo, ha bisogno di ricordare anche `last_action`, ovvero l'ultima azione eseguita, perché il segnale `bumped` arriva al passo successivo e senza questa informazione l'agente non saprebbe in quale direzione si trova appena il muro appena scoperto.

3.2 C2

L'agente si può classificare secondo le seguenti caratteristiche:

- **Osservabilità**: l'ambiente è parzialmente osservabile. L'agente percepisce solo la propria posizione, lo stato della cella corrente e il segnale di bump, ma non ha visibilità né sulle celle adiacenti né sull'estensione complessiva della griglia. Tutto ciò che si trova al di fuori della cella corrente è invisibile all'agente;
- **Conoscenza a priori**: l'agente ha una conoscenza a priori molto limitata. Conosce solo la propria posizione di partenza e le azioni disponibili, ma non conosce né le dimensioni della griglia, né la distribuzione dello sporco e neppure la posizione dei bordi. Ogni altra informazione viene acquistata durante l'esplorazione;
- **Difficoltà di pianificazione**: la pianificazione è significativamente più difficile rispetto al caso con dimensioni note. L'agente deve affrontare contemporaneamente due problemi distinti:
  - pulire le celle sporche;
  - esplorare uno spazio di cui non conosce i confini.

  Ciò costringe l'agente a bilanciare l'esplorazione e la pulizia ad ogni passo, senza poter pianificare un percorso ottimale in anticipo. Inoltre, la scelta casuale tra i candidati nella policy attuale può generare loop inutili, abbassando la performance a causa del costo di movimento applicato ad ogni passo.

3.3 C3

La politica attuale sceglie casualmente tra le celle non visitate, e quando tutte sono già visitate, torna su di esse generando loop inutili su celle già pulite, accumulando costi di movimento senza alcun beneficio.
L'idea è aggiungere all'insieme *visited* la distinzione tra celle visitate e celle sicuramente pulite, facendo in modo che l'agente le eviti finché esistono celle inesplorate. In pratica si aggiunge un insieme *clean* nello stato interno, che tiene traccia delle celle aspirate o trovate già pulite. L'agente preferisce nell'ordine: celle non visitate, poi celle visitate ma non sicuramente pulite, e solo in ultimo le celle pulite.

1. Inizializzazione dello stato: si aggiunge *clean* al dizionario restituito da *init_agent_2d_unknown_simple*.

```
def init_agent_2d_unknown_simple(start_pos: Position):
    return {
        "pos": start_pos,
        "visited": {start_pos},
        "walls": set(),
        "clean": set(),
        "last_action": None,
    }
```

2. Gestione *bump* e aggiornamento della posizione: la policy rimane invariata, ma ora si recupera anche *clean* dallo stato.

```
    pos = agent_state["pos"]
    visited = agent_state["visited"]
    walls = agent_state["walls"]
    clean = agent_state["clean"]
    last_action = agent_state["last_action"]
    (obs_pos, is_dirty, bumped) = percept
    # Gestione bump: segna il muro in base all'ultima azione
    if bumped and last_action in {"UP", "DOWN", "LEFT", "RIGHT"}:
        r, c = pos
        if last_action == "UP":    walls.add((r - 1, c))
        elif last_action == "DOWN":  walls.add((r + 1, c))
        elif last_action == "LEFT":  walls.add((r, c - 1))
        elif last_action == "RIGHT": walls.add((r, c + 1))
    pos = obs_pos
    visited.add(pos)
    if not is_dirty:
        clean.add(pos)
```

3. Scelta dell'azione con priorità di esplorazione: il cuore della modifica. L'agente ora distingue tre livelli di priorità prima di scegliere dove muoversi.

```
    if is_dirty:
        action = "SUCK"
        clean.add(pos)                # NUOVO: dopo SUCK e' sicuramente
            pulita
    else:
        neighbors = neighbors4(pos)

        # Priorita' 1: celle non visitate e non muro
        candidates = [(act, p) for act, p in neighbors.items()
                      if p not in walls and p not in visited]

        # Priorita' 2: celle visitate ma non sicuramente pulite
        if not candidates:
            candidates = [(act, p) for act, p in neighbors.items()
                          if p not in walls and p not in clean]  # NUOVO

        # Priorita' 3: celle pulite (ultimo resort)
        if not candidates:
            candidates = [(act, p) for act, p in neighbors.items()
                          if p not in walls]

        action = random.choice(candidates)[0] if candidates else "NOOP"
```

4. Aggiornamento e restituzione del nuovo stato: si propaga `clean` al passo successivo insieme agli altri elementi dello stato.

```
    r, c = pos
    new_pos = pos
    if action == "UP":      new_pos = (r - 1, c)
    elif action == "DOWN":  new_pos = (r + 1, c)
    elif action == "LEFT":  new_pos = (r, c - 1)
    elif action == "RIGHT": new_pos = (r, c + 1)

    new_state = {
        "pos": new_pos,
        "visited": visited,
        "walls": walls,
        "clean": clean,               # NUOVO
        "last_action": action,
    }
    return action, new_state
```

Effetto sulle performance: questa modifica riduce i loop inutili su celle già pulite finché esistono zone inesplorate, abbassando il numero di passi sprecati e quindi il costo di movimento accumulato. La performance risultante sarà generalmente più alta, soprattutto in ambienti grandi dove la politica casuale originale tende a rivisitare molto le stesse zone prima di esplorare quelle nuove. Lo svantaggio è minimo: in rari casi l'agente potrebbe impiegare qualche passo in più per tornare su una zona già visitata, ma il bilancio complessivo rimane positivo.

4 Mini-Report di Gruppo

4.1 Tabella comparativa dei quattro agenti

Agente / Proprietà	1D Reflex	2D Reflex	2D Model-Based parzialmente osservabile	2D Model-Based dimensioni ignote
Tipo di agente	Simple reflex	Simple reflex	Model-based	Model-based
Tipo di ambiente	1D lineare	2D griglia	2D griglia parzialmente osservabile	2D griglia parzialmente osservabile
Osservabilità	Pienamente osservabile	Pienamente osservabile	Parzialmente osservabile	Parzialmente osservabile
Conoscenza dimensioni	Sì (`n_squares` noto)	Sì (`rows` e `cols` noti)	Sì (`rows` e `cols` noti)	No (scoperte tramite bump)
Stato interno	`direction`	Nessuno	Mappa completa, posizione stimata	Mappa sparsa, `visited`, `walls`, `clean`, `last_action`
Funzione di performance	+1 per `SUCK`, costo movimento variabile	+1 per `SUCK`	+10 per `SUCK`, −1 per movimento	+10 per `SUCK`, −1 per movimento

Tabella 1.2: Tabella comparativa dei quattro agenti implementati.

La tabella mette in evidenza come la complessità degli agenti cresca progressivamente passando da sinistra a destra. I primi due agenti, **1D Reflex** e **2D Reflex**, sono *simple reflex*: non mantengono alcuno stato interno significativo e funzionano correttamente solo in ambienti pienamente osservabili, dove il percetto corrente è sufficiente a prendere decisioni ottimali.
La differenza tra i due risiede unicamente nella dimensione dell'ambiente, non nell'architettura.

Il passaggio ai due agenti **model-based** rappresenta un salto qualitativo: entrambi mantengono uno stato interno che permette di operare in ambienti parzialmente osservabili. La distinzione riguarda la *conoscenza a priori*: il primo conosce fin dall'inizio le dimensioni della griglia e costruisce una mappa completa, mentre il secondo scopre i confini tramite il segnale di bump, mantenendo una mappa sparsa che cresce durante l'esplorazione. Questo rende l'agente con dimensioni ignote il più complesso dei quattro, poiché deve affrontare simultaneamente la pulizia e l'esplorazione di uno spazio inizialmente sconosciuto.

4.2 Discussione sulla razionalità nei diversi contesti

L'agente razionale seleziona un'azione che massimizza la performance.

1. **L'agente 1D Simple Reflex** opera in un corridoio monodimensionale, il cui ambiente è completamente osservabile e le dimensioni sono note; di conseguenza, la percezione corrente contiene le informazioni necessarie per prendere la decisione più ottimale possibile. Ciò rende l'**agente razionale**, che non perde le informazioni agendo solo sulla percezione immediata.

2. **L'agente 2D Reflex** opera in una matrice. Anche se il suo ambiente è completamente osservabile, tuttavia non ha memoria delle celle già visitate; quindi, rischia di esplorare percorsi già visitati, lasciando sporche alcune porzioni della matrice. Questo tipo di agente **non è razionale**, infatti, non massimizza il valore atteso della sua performance.

3. **L'agente model-based** che opera in un ambiente parzialmente osservabile è **razionale**. Mantiene lo stato interno, integrando le percezioni passate con quelle attuali e ricostruendo una rappresentazione dell'ambiente delle parti inosservabili. In questa maniera, pianifica una copertura completa della griglia, massimizzando la performance nonostante la visibilità limitata.

4. **L'agente 2D Model-Based con dimensioni ignote** è **razionale** ma limitato. Inizialmente, non conosce le dimensioni dell'ambiente, costruisce la propria mappa progressivamente esplorando l'ambiente. Questo agente soddisfa il criterio di razionalità perché agisce sempre nel modo migliore possibile data la sua conoscenza.

4.3 Estensioni possibili

- **Sporco stocastico**: se lo sporco potesse ricomparire in modo stocastico, il problema non sarebbe più episodico ma continuo: l'agente non potrebbe mai considerare il proprio compito terminato, poiché una cella appena pulita potrebbe sporcarsi nuovamente in qualsiasi momento.

Il cambiamento più importante riguarderebbe lo **stato interno**: l'insieme `clean` perderebbe di utilità, poiché una cella segnata come pulita non offrirebbe più alcuna garanzia sul suo stato futuro. L'agente dovrebbe quindi rivisitare periodicamente tutte le celle, anche quelle già pulite. Di conseguenza, la funzione di performance cambierebbe: non avrebbe più senso misurare il successo come raggiungimento di uno stato finale "tutto pulito", ma piuttosto come qualità media dell'ambiente nel tempo, premiando l'agente per mantenere basso il numero di celle sporche in modo continuativo. Un agente razionale in questo contesto dovrebbe bilanciare la frequenza di rivisitazione di ciascuna cella con il costo di movimento, ad esempio controllando l'ambiente in modo ciclico con una frequenza proporzionale alla probabilità di ricomparsa dello sporco.

- **Ambiente multi-agente**: introdurre un secondo aspirapolvere nello stesso ambiente causa una serie di problemi:
  - **coordinamento**: se i due agenti si muovono in modo indipendente e casuale, rischiano di pulire ripetutamente le stesse celle ignorando zone ancora sporche, riducendo l'efficienza complessiva. Un design razionale dovrebbe prevedere una forma di divisione dello spazio, ad esempio assegnando a ciascun agente una metà della griglia, oppure un meccanismo di comunicazione che permetta ai due agenti di condividere la mappa esplorata e lo stato delle celle.
  - **gestione dei conflitti**: due agenti potrebbero tentare di occupare la stessa cella simultaneamente, rendendo necessaria la gestione del simulatore. Inoltre, se entrambi aspirano la stessa cella nello stesso passo, uno dei due esegue un'azione inutile accumulando comunque il costo di movimento. Dal punto di vista del design, si passerebbe da un sistema single-agent a un sistema multi-agente cooperativo, in cui la funzione di performance è condivisa e l'obiettivo è massimizzare la pulizia collettiva dell'ambiente nel minor numero di passi totali possibile.